CST2550

Group Coursework

Design a Data Structure for Effective Loading and Saving Data from SQL Database

Codeflix

Student Details

Siddharth Joshi- M01054309 (Team Leader)

Vansh Roneshkumar Patel - M01052925 (Secretary)

Om Sarvaiya - M01031652 (Developer)

Arvind Choudhary - M01050641 (Developer)

Zaqwan Abdurrasid Sidat - M01047548 (Tester)

Table of Contents

1. Introduction 4
2. System Design 4
   2.1. Data Structure Design 4
   2.2. Justification for Using a BST 4
   2.3. System Architecture 5
   2.4. Core Operations 5
      2.4.1. Insert Operation 5
      2.4.2. Search Operation 6
      2.4.3. In-Order Traversal 6
   2.5. Time Complexity Analysis 6
3. Implementation Overview 6
   3.1. Core Components 6
   3.2. Functional Workflow 7
   3.3. Database Integration 7
   3.4. Error Handling 8
4. Testing 8
   4.1. Testing Methodology 8
   4.2. Test Cases 8
   4.3. Validation Testing 9
   4.4. Results and Findings 10
5. Conclusion 10

References 11

Appendix: Contribution Information 12

List of Figures

Figure 1: System Architecture 5
Figure 2: Time Complexity Table 6
Figure 3: Class Diagram 7
Figure 4: Test Cases Description and Results Table 9

1. Introduction

Digital Media, such as video and music files, have been growing in popularity and there has also been a need to find a better way to store and retrieve the information about movies and TV programs. Existing ways of doing this (that is, traditional means of storing the data) do not provide adequate means for searching or maintaining the organization of large amounts of media records, but we will attempt to address this issue by presenting the development of **Codeflix**, a Movie/TV Library Search System created in C#.

The primary goal of **Codeflix** is to provide an efficient way to store, manage and search for media records that are organized well. The database is structured using Binary Search Trees (BST), which allow for fast insertion, deletion, and retrieval of records in the system. By storing the data in this manner, the user will have an easier time being able to search for media by title.

Codeflix also allows for both in memory and SQL storage of records, allowing users to save their records to be retrieved later when the program is run.

Using a combination of Binary Search Trees and SQL, **Codeflix** allows for the design of a system that has demonstrated real world application of the software engineering principles, as well as allowing for both in-memory and persistent storage of media records (Sommerville, 2016).

In summary, this project will show how to utilize data structures and a database system and create a functional application for users.

2. System Design

Codeflix is architecturally modeled following modular design principles using a data structure combination consisting of a custom coded data structure along with a relational database for persistent storage. In the core design an in-memory representation of media records is constructed using a Binary Search Tree (BST), and media records are kept permanently in a relational database.

2.1. Data Structure Design

The BST is used as the data structure for managing media records in Codeflix. A BST is defined as a data structure in which a node's children are orderly represented according to an ordering rule such as lexicographically (Cormen, et al., 2022). For example, given a root node in a BST, all child nodes on the left (i.e., all media items in the left subtree) have a lexicographical title value which is less than (i.e., smaller than or equal to) the title of the root node; and all child nodes to the right (i.e., media items in the right subtree) have a lexicographical title value which is greater than (i.e., larger than) the title of the root node. The BST provides efficient operations including searching, inserting and deleting while simultaneously maintaining the inherent sorted structure of the media records.

2.2. Justification for Using a BST

There are several reasons the BST was selected over other data structures.

1. Efficient Searching: The ability of a BST to provide fast lookups relative to linear data structures is an important feature of BSTs.
2. Sorted Output: All media records returned by an in-order walk of a BST returned in a sorted order.

3. Dynamic Structure: The ability of a BST to support both the insertion and deletion of media records dynamically without the need for resizing is a significant advantage.
4. One's Personal Education: Using a BST is an excellent means of demonstrating one's understanding of fundamental data structures.

While an array or linked list is relatively simple compared with a BST, an array or linked list typically necessitates additional mechanisms to obtain a sorted output and therefore typically will provide slower search performance compared with a BST. More advanced data structures (e.g., a hash table) do not retain their order and therefore are not an appropriate choice for use in Codeflix. Balanced trees (e.g., AVL trees) introduce significant additional complexity which is outside the scope of the Codeflix system design project.

2.3. System Architecture

The system consists of three main components

1. User Interface
2. Business Login Layer
3. Database Layer

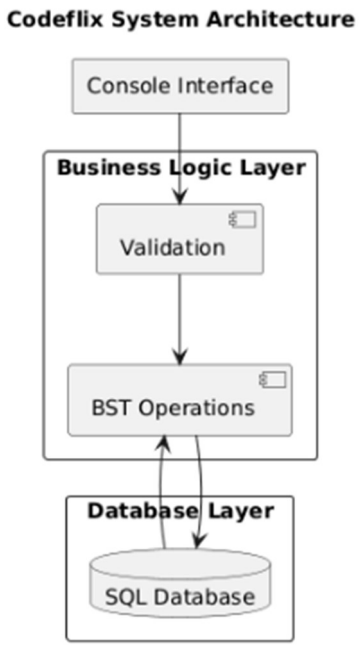

Figure 1: System Architecture

The layered approach helps in maintainability of the project.

2.4. Core Operations

2.4.1. Insert Operation

```
IF root is null THEN
   create new node
ELSE IF value < root THEN
   insert into left subtree
```

```
ELSE
    insert into right subtree
```

2.4.2. Search Operation

```
IF root is null OR value equals root THEN
    return result
ELSE IF value < root THEN
    search left subtree
ELSE
    search right subtree
```

2.4.3. In-Order Traversal

```
Traverse left subtree
Visit node
Traverse right subtree
```

2.5. Time Complexity Analysis

Operation	Average Case	Worst Case
Insert	O(log n)	O(n)
Search	O(log n)	O(n)
Delete	O(log n)	O(n)

Figure 2: Time Complexity Table

BST has best performance for average case, while for worst case, the performance goes to linear time.

3. Implementation Overview

This project is a Console application developed in C#. The implementation is done by integrating Binary Search Tree logic and SQL Database connection to create a functional system.

3.1. Core Components

The proposed system consists of several core components:

- **MediaItem Class**
  Represents a movie or TV show and has the following attributes: title, genre, release year, and rating.
- **BinarySearchTree Class**
  Contains the core BST functions, including insertion, searching, deletion and in-order traversal which allows the records to be displayed in a sorted manner.
- **Validation Module**
  Validates all user input (e.g. ensuring that the title field is not empty and that the numeric value is within valid range for year and rating) to ensure the reliability of the application.
- **Database Handler**
  Handles the communication between the application and the SQL database for saving records and retrieving stored records.

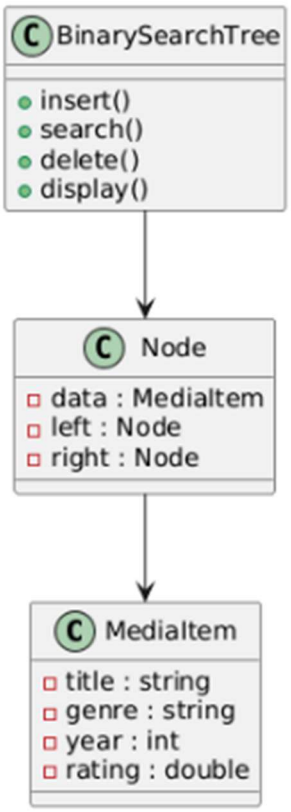

Figure 3: Class Diagram

3.2. Functional Workflow

The application is used through a menu driven system, allowing the user to perform operations against the application. The general workflow of the application is as follows:

- The user selects an operation to perform from the menu.
- The input is validated before the application processes it.
- The application performs the BST operations in memory.
- The application will save the data to the SQL database and/or retrieve the data from the database if the user desired the option.
- The application displays the results to the user.

This workflow provides a smooth user experience and provides a consistent means to manage data.

3.3. Database Integration

The application connects to a SQL Server database using the connection string. The database will allow the Codeflix application to permanently store the media records, therefore allowing the application to:

- To save all of the records the user has added to the database.
- To load all of the records that the user previously added to the BST.

- To ensure that the user's data will remain consistent across all applications that are using the same SQL database.

The application uses standard SQL queries to insert and retrieve data from the database ensuring that the communication between the application and database is efficient.

3.4. Error Handling

Error handling is done in a way to make the application robust.

- Invalid input will be rejected and an appropriate response provided.
- Database connection issues are handled in a graceful way.
- Edge case (like search on empty tree); handled as required.

4. Testing

In order to establish that the Codeflix platform is producing accurate data and is dependable and has a solid structure the development team, conducted a combination of unit tests and manual tests (Myers, et al., 2004). During these tests the development team reviewed the functionality of the all major components of the Codeflix system to provide for proper validation of the different elements.

4.1. Testing Methodology

The testing of the Codeflix system examined all critical functions. The following items were reviewed for core functionality, which are:

- Binary Search Tree operations, including but not limited to: insert, search, delete and traverse
- The functions for validating input
- Integration of the database such as save and load functions

The development team conducted unit tests to validate each component individually, and conducted man tests to validate the overall system and the end-user-interaction with the Codeflix system.

4.2. Test Cases

The table below contains all the text cases used to validate the system.

Test Case ID	Component	Test Description	Input Example	Expected Output	Result
TC01	MediaItem	Verify constructor sets all properties correctly	MediaItem(1, "Inception", "Sci-Fi", 2010, "Movie", 8.8, "Christopher Nolan")	All properties stored correctly	Pass
TC02	MediaItem	Verify ToString() returns	MediaItem("Dark", "Sci-Fi", 2017, "TV Show")	Returned string contains	Pass

		formatted content		title, genre, year, and type	
TC03	BST Insert	Insert a new unique media item	"Dune"	Item inserted successfully and searchable	Pass
TC04	BST Insert	Reject duplicate title insertion	"Inception"	Insert returns false	Pass
TC05	BST Search	Search for an existing title	"Inception"	Matching media item returned	Pass
TC06	BST Search	Search for a non-existing title	"Nonexistent Title"	Null returned	Pass
TC07	BST Delete	Delete an existing title	"Dark"	Item removed successfully and no longer searchable	Pass
TC08	BST Delete	Attempt to delete a non-existing title	"Unknown"	Delete returns false	Pass
TC09	BST Traversal	Return all items in sorted order	Existing tree with 4 items	Items returned alphabetically: Breaking Bad, Dark, Inception, The Matrix	Pass
TC10	BST Clear	Remove all items from the tree	Clear() on populated tree	Tree becomes empty and item count becomes 0	Pass

Figure 4: Test Cases Description and Results Table

4.3. Validation Testing

Through the validation tests it was discovered that using an incorrect or invalid data type does not negatively affect the overall performance of an application. In order to ensure this the development team validated the following items:

- Empty or null titles will be rejected
- Numeric values will be validated to ensure that each value is appropriately within the boundaries of the other values (i.e. the year and the rating will not be less than zero)
- Any invalid input will return an appropriate response

When the above items are tested and validated there will be an assurance of data integrity and no run-time errors.

4.4. Results and Findings

All of the above listed validation tests were successful. There were no unexpected deviations from the validity of the BST based on the results of the validation tests (the BST maintains sorted order) and there were no unexpected errors encountered for the integration of Codeflix to the database. All of the bugs and deficiencies coded during the actual development phase were corrected.

The overall results indicate that Codeflix has demonstrated stability, functionality, and is ready for deployment.

5. Conclusion

The Codeflix project has shown how to use database integration and data structures in a real-world situation by using a Binary Search Tree (BST) for the storage, searching and retrieving media record data. The BST method of storing media record data uses an organized method of keeping media records in a way that allows for efficient use of space while maintaining an organized method of keeping media records in sorted order. The integration of using a SQL database also allowed for persistent storing of data so that once entered into the Codeflix database, it remains available to the Codeflix users for future access, meaning a user could enter media records into the CODEFLIX database during one login, and those same media records would still be available the next time the same user logs into CODEFLIX. Validation and Unit Testing also improved the accuracy and reliability of the Codeflix application. Overall, Codeflix has met all of its goals for this project scope and time frame. Areas for future improvement may include implementing a self-balancing tree structure and adding a graphical user interface (GUI) to improve usability.

References

Cormen, T., Leiserson, C., Rivest, R. & Stein, C., 2022. *Introduction to Algorithms..* s.l.:MIT press.

Myers, G., Badgett, T., Thomas, T. & Sandler, C., 2004. *The art of software testing (Vol. 2)..* Chichester: John Wiley & Sons..

Sommerville, I., 2016. *Sommerville, I..* 10th ed. s.l.:Pearson.

Appendix: Contribution Information

Commits on Mar 21, 2026

Added Video Demonstration File
siddharthjoshi1144 authored 4 minutes ago

Added Readme File - Added How to Run File - Added Daily Scrum File - Added Team Roles Discription and Task Division
siddharthjoshi1144 committed 5 minutes ago

Added Meeting Minutes
Vansh-Roneshkumar-Patel committed 12 minutes ago

10 Unit Tests Added
Zaqwan-Abdurrasid-Sidat committed 15 minutes ago

Code Finalized
Arvind-Choudhary-UK committed 21 minutes ago

Integrated SQL Database and Added Input Validation Code
Arvind-Choudhary-UK committed 25 minutes ago

Added Basic Logic and Logic for Binary Search Tree
Om-Sarvaiya committed 29 minutes ago

Initial commit - Repository Created
siddharthjoshi1144 committed 1 hour ago